

Homework 10: A Config Loader That Tells The Truth

Build something that fails in several ways, then reproduce three anti-patterns so you have written each one at least once deliberately rather than by accident.

Part One: The Loader

Create `ConfigLoader.java` in the classic form.

Write a `load(String name)` method returning a `Map<String, String>` parsed from `key=value` lines. Simulate three files:

Name	Behaviour
<code>app.conf</code>	returns <code>host=localhost</code> and <code>port=8080</code>
<code>missing.conf</code>	the reader throws <code>IOException</code>
<code>broken.conf</code>	parses, but <code>port</code> is not a number

Write a `ConfigException` that:

- is **checked**, with a comment saying why checked is the right call here
- carries the **source file name** as a field, not glued into the message
- has a constructor taking a `Throwable` cause

Load all three files in `main`. For each failure, print the message, the source, and the cause.

The `broken.conf` case is the interesting one. `Integer.parseInt` throws an *unchecked* `NumberFormatException`. Decide whether to let that escape as-is or convert it, and justify your choice in a comment.

Part Two: Three Anti-Patterns

Write each of these twice, once broken and once fixed, and print both results.

1. The empty catch block. Catch something and do nothing. Then fix it.

In a comment, explain in your own words why this is worse than letting the program crash.

2. `return` inside `finally`. Throw from the `try`, `return` from the `finally`, and show that the exception vanishes. Then fix it.

Explain how this is the empty catch block wearing different clothes.

3. Catching `Exception` too broadly. Write a method that *intends* to handle a parsing failure but whose real bug is a `NullPointerException`, and show the handler misreporting it. Then fix it.

Explain what the broad catch cost whoever has to debug it.

Part Three: Resources

Open two `AutoCloseable` resources in one try-with-resources. Make **both** `close()` methods throw, and make the **body** throw as well.

Catch the result and print the caught exception plus everything from `getSuppressed()`.

Then write a comment covering:

1. Which resource closed first, and why that order is the correct one
2. Which exception you caught, and what happened to the other two
3. What hand-written cleanup in a `finally` block would have done instead

Acceptance Criteria

- ☐ Runs with `java ConfigLoader.java`
- ☐ `ConfigException` is checked, carries the source as a field, and has a cause constructor
- ☐ All three config cases are exercised and print their cause
- ☐ A comment justifies checked over unchecked for `ConfigException`
- ☐ A comment justifies the `NumberFormatException` decision
- ☐ All three anti-patterns appear broken and fixed, with output showing the difference
- ☐ Each anti-pattern has an explanation in your own words
- ☐ The try-with-resources output shows reverse close order and two suppressed exceptions
- ☐ No empty catch block survives in the final version except the one deliberately labelled as the anti-pattern

Stretch

Rewrite `load` so it never throws at all, returning a sealed `Result` type instead, as Module 09's stretch did.

Then write a short comparison. Which is easier for the caller to get wrong? Which reads better at the call site? Which one survives being called from a lambda inside a stream, and why does that matter given what Module 10 said about checked exceptions and the streams API?

Hint, if the suppressed list comes back empty

Suppressed exceptions only appear when the **body** throws as well. If the body completes normally and a `close()` throws, that close exception becomes the primary one and there is nothing to suppress it against.